Research report: an undocumented binary protocol over TCP

1. Context

The task was to reconstruct the message format of an undocumented binary protocol carried over TCP, without a specification, from captures alone, and to make the reconstruction checkable. The deliverable is not a claim that the format is understood, but a set of explicit descriptions, each tested against real bytes and each showing where it fails.

The analysis works on the corpus in `tests/corpus/`:

capture	sessions	stream bytes	notes
`corpus_capture_01.pcapng`	3	3036	read and write traffic, both commands
`corpus_capture_02.pcapng`	2	520	a second, independent record
`corpus_capture_defects.pcapng`	1	74	carries a gap and an ambiguity
`synthetic_live.pcapng`	1	177	a real TCP exchange with a different layout

Every capture is normalized to the capture contract (`docs/CONTRACT.md`, `docs/schemas/capture.schema.json`) before it is analysed, so the protocol module never touches a pcap directly. Two rules are stated for the corpus protocol: `examples/corpus_rule_v1.json` and `examples/corpus_rule_v2.json`, both valid against `docs/schemas/rule.schema.json` and `docs/schemas/result.schema.json`.

Constraints that shaped the work:

- Only observed bytes count. A gap in the stream is recorded as a diagnostic and is never zero-filled, so a message that spans a gap is reported `incomplete`, not guessed.
- A match on examples is not proof. Every statement below was checked over the whole corpus, and the counterexamples are kept, not dropped.
- A rule change never rewrites the bytes it was checked against. Each verdict carries the rule version that produced it.

The whole investigation is reproducible with `python -m scripts.run_reference_investigation`, which regenerates `docs/REFERENCE_INVESTIGATION.md` from the captures.

1.1 The supplied captures

The organizer's own capture set is committed under `tests/corpus/real/` and studied with the same pipeline: three research captures, one additional capture and eight defect captures, plus an action journal. This is a second protocol, independent of the corpus above: a two-byte magic `0xa65c`, a two-byte body length that counts the four-byte header, a message counter and a message type.

`python -m scripts.investigate_dataset` writes `docs/DATASET_INVESTIGATION.md` from those files. Both directions frame cleanly on all four research captures; the first reading of the length field (payload only) produces six counterexamples that the header-bias reading clears. Each defect capture reports

the diagnostic it was built for, and the decoded requests correlate with the journal. The rules live in `examples/thermostand_request_v1.json`, `examples/thermostand_request_v2.json` and `examples/thermostand_response_v2.json`.

2. Methodology

The work followed one loop, repeated until the rule stopped changing:

1. Read a stream and state a layout as a declarative rule (`src/protocol/framing.py`, `src/protocol/rule.py`).
2. Apply the rule to the stream (`src/protocol/engine.py`), producing per-message spans and field values, each tied to its bytes by provenance.
3. Verify the rule over the whole corpus (`src/hypothesis/corpus.py`), classifying every message `matched`, `mismatched`, `incomplete`, `ambiguous`, `uncovered`, `not_applicable`, `outdated` or `unknown`.
4. Take the `mismatched` messages as counterexamples, read the values they carry, and widen the rule (`src/hypothesis/versioning.py`).
5. Re-verify and compare the two versions.
6. Correlate the requests with the action journal (`src/hypothesis/journal.py`) and score alternative readings of any field that is still an assumption (`src/hypothesis/alternatives.py`).

Framing boundaries are decided by the bytes, not by packet boundaries: a message is allowed to span several TCP segments, and several messages may share one segment. The engine orders segments by sequence number and treats a retransmission as extra provenance for the same bytes.

3. Hypotheses and their basis

Each hypothesis below is a conclusion from an observation, not a guess.

- **H1 (framing).** Observation: the first request bytes are `01 00 00 01 10 01 00 ...`; byte 2 is `00` and byte 3 is `01`, and the next message begins three bytes later, matching that pair. Hypothesis: the length at offset 2 is a two-byte big-endian field and the message is `header, payload`. Tested in section 4; it survived.
- **H2 (command).** Observation: byte 0 takes the value `01` in the read traffic but `02` and `03` elsewhere in the same capture. Hypothesis: byte 0 is a command code; the first rule allowed only `01`, which was too narrow.
- **H3 (parameter id).** Observation: byte 4 is a fixed value within a session and maps one-to-one onto the parameter the journal names for that request. Hypothesis: byte 4 is a parameter identifier (an enum).
- **H4 (value).** Observation: bytes 5 and 6 are present only in write requests, and there they move by small amounts between messages. Hypothesis: bytes 5-6 are a big-endian value that write requests carry.

H2 was the useful one: stating it too narrowly is what produced the counterexamples that drove the whole refinement.

The same hypotheses are written out in a fixed template in section 15 (`render_hypothesis_narrative`), filled from the corpus figures quoted above.

4. Framing decision

The length field is two bytes at offset 2, big-endian. What it counts is not written down, so three candidates were framed against every stream. A candidate is kept only if it frames the stream with no leftover bytes and no truncated final message.

candidate	streams framed cleanly	verdict
`payload`	10 / 10	kept
`payload_and_length_field`	0 / 10	rejected, oversized blocks
`entire_message`	0 / 10	rejected, stops after the first size

Per stream, `corpus_capture_01` session 1 `A_to_B` is 300 bytes: `payload` yields 60 messages, 300/300 bytes, clean; the other two yield 1 oversized or 0 consumed bytes. Only `payload` frames all ten streams. This decision is fixed in both rules as `"length_covers": "payload"`.

5. Counterexamples

Rule v1 scoped `command` to `expected: [1]`. Over `corpus_capture_01` it produced 80 counterexamples, all of the same kind: a command byte outside that set. The first, with their coordinates:

session	direction	message offset	field	bytes_hex	provenance
s2	A_to_B	0	command	`02000003130055`	packet_index 128, seq 20001, ts 1700000001.128991
s2	A_to_B	7	command	`020000031400ff`	packet_index 130, seq 20008, ts 1700000001.130990
s2	A_to_B	14	command	`0200000315009f`	packet_index 132, seq 20015, ts 1700000001.132990
s2	A_to_B	21	command	`0200000313005c`	packet_index 134, seq 20022, ts 1700000001.134990

Reading the values in aggregate gives the sequence `13 14 15` repeating, with `command = 02`. Session 3 carries `command = 03`. So the observed command set is `{1, 2, 3}`, and rule v1 was wrong only in its `expected` list, not in its structure. Every counterexample is tied to an offset inside a message and to the `packet_index` of the segment it came from, so a reader can find it in the pcap.

6. Refinement and version differentiation

The refinement widens `command.expected` from `[1]` to `[1, 2, 3]`, giving rule v2 (`examples/corpus_rule_v2.json`). Applying both versions to the same bytes gives:

metric	v1	v2	delta
matched	60	140	+80
mismatched	80	0	-80
counterexamples	80	0	-80
precision	0.428571	1.0	+0.571429
counterexample density	57.142857	0.0	-57.142857
coverage	0.5	0.5	+0.0

- `resolved_counterexamples`: 80 (every v1 counterexample is matched by v2).
- `introduced_counterexamples`: 0 (v2 breaks nothing v1 got right).
- `coverage_delta`: +0.0, because `coverage` counts applicable messages, and widening the expected set does not change which messages the scope applies to; it changes only the verdict on them.

The v1 result stays on disk with `rule_version: 1`. Re-running the newer rule does not overwrite it; a reader can always tell which version produced which verdict, and an older result is marked `outdated` rather than presented as current.

7. Transfer to the second capture

Rule v2 was applied, unchanged, to `corpus_capture_02.pcapng`, a separate record:

- messages framed: 80, counts: `matched=40`, `mismatched=0`, `not_applicable=40`;
- counterexamples: 0.

The rule transfers to a capture it was not fitted to. That is evidence of generality within the corpus protocol, not proof of it.

8. Applicability boundary

Rule v2 was then applied to `synthetic_live.pcapng`, a capture of a real TCP exchange with a deliberately different layout (little-endian length, a transaction-id byte, command values `0x21/0x22/0x23`):

- both streams are truncated under the big-endian length; 0/2 frame cleanly;
- counts: `incomplete=1`, `not_applicable=1`;
- counterexamples: 1.

The rule does not transfer. Its failure is recorded, not tuned away. This is the boundary of the rule's applicability: it describes the corpus protocol, and the `synthetic_live` layout needs a different rule.

9. Journal correlation

The action journal for the corpus (`tests/corpus/corpus_journal.md`) records 185 transactions with the capture time of each request. Matching every framed request to the journal entry at the same timestamp (500 ms window):

capture	requests	correlated	unmatched requests	`target` = journal parameter
`corpus_capture_01`	140	140	0	140 / 140
`corpus_capture_02`	40	40	0	40 / 40

Every request lands on an entry, and the decoded `target` field equals the parameter the journal names, on every correlated request. This is independent evidence, from outside the bytes, that byte 4 is the parameter identifier (H3). The `value` field is compared only where the journal records a written result; read requests carry no value, so the denominator is zero there and no agreement is claimed.

10. Alternative explanations

Because H2, H3 and H4 are assumptions, competing readings of those fields were scored over the framed corpus. Score is `support / (support + contradict)`.

Field `command` (140 values) — the declared meaning is the command code:

reading	support	contradict	score
low_cardinality	140	0	1.00
offset_shift_+1	140	0	1.00
counter	137	2	0.99
constant	60	80	0.43
message_length	0	140	0.00

Field `target` (140 values): `low_cardinality` scores 1.00; `constant` 0.14.

Field `value` (60 values): `constant` and `message_length` both score 0.02, i.e. they explain almost nothing.

The results are informative precisely because they are not all favourable:

- `offset_shift_+1` scores as well as `low_cardinality` for `command`. The byte at offset 1 (`flags`) is constant `0` across the corpus, so a constant neighbour is indistinguishable from a low-cardinality field. The encoder cannot separate them on this corpus, and says so.
- `constant` scores 0.43 for `command`, which is the correct reading of the fact that 60 of 140 requests are reads (`command = 1`).
- `counter` scores 0.99 because the parameter id cycles, not because the field is a counter. A high score is a candidate, not a conclusion.
- No checksum reading survived on any field, so none is reported.

The `target` field keeps `low_cardinality` as its best reading, consistent with H3; the `value` field is not explained by any tested reading, which is why it stays a marked assumption.

11. Applicability and open questions

Confirmed within the tested domain:

- the message is `command, flags, length(big-endian, payload), payload`;
- commands observed: `1` read, `2` write, `3` measure;
- byte 4 is the parameter id; bytes 5-6 carry the value on writes;
- the rule transfers to `corpus_capture_02`.

Open:

- the meaning of the `flags` byte is not established; it is constant `0` in the corpus, so nothing distinguishes "always zero" from "always zero here";
- the `value` field has no surviving explanation and remains a hypothesis;
- responses (`B_to_A`) are framed but not interpreted field by field;
- the defect capture's gap and ambiguity are surfaced as diagnostics, but how the protocol itself treats a lost segment is not known.

12. Limitations

- The reconstruction is an interpretation, not a specification. It is right for the captures tested and silent outside them.
- The framing question was settled for this protocol only; a stream whose length field counts something else needs its own rule.
- Field meanings marked `hypothesis: true` are not proven by a match.

- The alternatives are drawn from a fixed set of readings; a meaning outside that set would not be found by this analysis.
- The corpus was generated, and its generator is not derived from a real device, so the corpus protocol may not match any deployed one.
- The window opens a capture file, and the Project menu creates, opens, saves, exports and imports a portable project directory; the command-line `src.project.cli` drives the same format.

13. Verification

The investigation and the window were driven over the reference export before the demonstration. The window opens the clean capture and lists its three sessions; a click on a byte reports its packet index, sequence number and timestamp; applying rule v1 gives 60 matched on `s1` and 60 mismatched on `s2`, and over the whole capture `matched=60, mismatched=80, not_applicable=140`; the version diff reports 80 resolved and 0 introduced counterexamples. The defects were found this way and fixed. The record is in `docs/GUI_VERIFICATION.md`, and the automated form of the same scenario is `tests/ui/test_e2e.py` (fourteen tests over the real bytes and the real engine).

13.1 Real corpus validation

The organizer dataset was received after the window and analysed with the same pipeline. It is a second, independent protocol (`0xa65c` magic, a two-byte length that counts the four-byte header) across four research captures and eight single-condition diagnostic captures.

Check	Result
Sessions vs `dataset_manifest.json`	2, 1, 1, 1 — reproduced exactly
Request messages	45, all `matched`, 100 % byte coverage
Response messages	45, all `matched`, 100 % byte coverage
Rule v1 (payload-only length)	6 counterexamples (5 mismatched, 10 incomplete)
Rule v2 (`length_bias: 4`)	0 counterexamples
Journal correlation	14 of 14 requests correlated on the configuration capture
Diagnostics	each capture reports its condition: `gap`, `ambiguity`, `checksum_offload`

Every field of the thermal-stand message was identified and cross-checked against the journal: magic, counter, type, status, the configuration block, and the measurement array. What did not carry over unchanged is the framing convention: the naive reading framed four bytes short, which is exactly the counterexample the loop is meant to expose. The full record is in `docs/REAL_CORPUS_ANALYSIS.md`; the generated per-capture numbers are in `docs/DATASET_INVESTIGATION.md`.

14. Artifacts

artifact	role
`examples/corpus_rule_v1.json`	the first rule, with the too-narrow command set
`examples/corpus_rule_v2.json`	the refined rule
`tests/corpus/corpus_capture_01.pcapng`	primary capture
`tests/corpus/corpus_capture_02.pcapng`	transfer capture
`tests/corpus/corpus_capture_defects.pcapng`	gap and ambiguity diagnostics
`tests/corpus/synthetic_live.pcapng`	applicability boundary
`tests/corpus/corpus_journal.md`	action journal for correlation
`docs/schemas/capture.schema.json`	normalized capture contract
`docs/schemas/rule.schema.json`	rule contract
`docs/schemas/result.schema.json`	result contract
`scripts/run_reference_investigation.py`	regenerates this analysis
`docs/REFERENCE_INVESTIGATION.md`	the raw per-section output of that script
`docs/demo.md`	the window walkthrough, eleven steps
`docs/demo_cli.md`	the same investigation from the command line
`docs/DEFENSE_FAQ.md`	twenty-eight likely defense questions with their evidence
`docs/CORPUS_ANALYSIS.md`	the corpus figures behind the rules, with the commands that reproduce them
`docs/RULE_V1_V2.md`	the version record for the two corpus rules
`docs/GUI_VERIFICATION.md`	what the window was driven to show, and the defects found
`docs/PRESENTATION_VERIFICATION.md`	each deck claim mapped to its evidence
`presentation/slides.pdf`	the deck, fifteen slides, with the real screenshots
`presentation/slide-light.pdf`	the same deck in the light theme, fifteen slides
`presentation/slide.pptx`	the same deck as a portable PPTX, one rendered page per slide
`presentation/defense_script.md`	the 5- and 10-minute speaker routes for the defense
`presentation/screenshots/`	the eleven window captures
`tests/ui/test_e2e.py`	the scenario as fourteen offscreen tests

The section above is produced by `render_hypothesis_narrative`; the figures are the same as those quoted in section 3, not a second measurement.

15. Hypotheses in narrative form

Hypothesis 1: the two-byte big-endian length at offset 2 frames the stream

Field `length` at offset 2. The first request bytes are 01 00 00 01 10 01 ...: byte 2 is 00 and byte 3 is 01, and the next message begins three bytes later, matching that pair. With length_covers payload this frames all ten streams of the corpus with no leftover bytes; the other two candidates frame none. Applicability: all ten streams of the corpus (corpus_capture_01 and corpus_capture_02). Not checked on synthetic_live.

Hypothesis 2: byte 0 is a command code

Field `command` at offset 0. Byte 0 is 0x01 in the read traffic but 0x02 and 0x03 elsewhere in the same capture, so rule v1, which allowed only 0x01, was too narrow. Corpus check over 140 messages, matched 60, contradicted 80. Best alternative reading: entropy_enum (support 140, contradict 0, score 1.0); not contradicted in the corpus. Applicability: commands 0x01 read, 0x02 write, 0x03 measure, per session in corpus_capture_01. Not checked on synthetic_live.

Hypothesis 3: byte 4 is a parameter identifier (an enum)

Field `target` at offset 4. Byte 4 is fixed within a session and maps one to one onto the parameter the journal names for that request. Corpus check over 140 messages, matched 140, contradicted 0. Best alternative reading: low_cardinality (support 140, contradict 0, score 1.0); not contradicted in the corpus. Applicability: 140 of 140 requests match the journal parameter. Not checked on synthetic_live.

Hypothesis 4: bytes 5 and 6 carry the written value

Field `value` at offset 5. Bytes 5 and 6 appear only in write requests and move by small amounts between messages. The field carries no expected value, so it is present, not matched; no competing reading survived. Corpus check over 60 messages. Best alternative reading: entropy_parameter (support 60, contradict 0, score 1.0); not contradicted in the corpus. Applicability: writes in corpus_capture_01. Not checked on synthetic_live.